ECE456 Network Centric Programming – Midterm

1. (5 points) Dead code and optimization.

   Consider the following program compiled **without** optimization:

```c
int main() {
    long long sum = 0;
    for (long long i = 0; i < 1000000000LL; i++) {
        sum += i;
    }
    return 0;
}
```

   On this machine the program takes **1.5 seconds** to run.

   If we compile with optimization how long does it take (rounded to the nearest 0.1 seconds)?

   Time: _____ seconds

   Now consider:

```c
#include <unistd.h>

int main() {
    for (int i = 0; i < 1000; i++) {
        usleep(1000); // 1000 microseconds = 1 ms
    }
    return 0;
}
```

   Roughly how long does this program take to run (assume `usleep()` is exact)?

   Time: _____ seconds

   Next:

```c
int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    int c = a + b;
    return 0;
}
```

   At runtime, is an integer addition performed? _____

   Finally, consider:

```c
int add(int a, int b);

int main() {
    int c = add(2, 3);
    return 0;
}

int add(int a, int b) {
    return a + b;
}
```

With normal compilation (no `inline`), is an addition performed at runtime? _____

If we mark `add()` as `inline` and enable optimization, can the compiler remove the addition and replace it with a constant?

_____

2. (5 points) For each program, write what happens.

```
#include <sys/stat.h>
#include <sys/types.h>
#include <iostream>
using namespace std;
int main() {
    int status =chdir("xyz");
    cout << "status: " << status << endl;
    return 0;
}
```

Net effect of the program: _____

```
#include <sys/stat.h>
#include <sys/types.h>
#include <iostream>
using namespace std;
int main() {
    int status =mkdir("xyz", 0755);
    cout << "status: " << status << endl;
    return 0;
}
```

Net effect of the program: _____

```
#include <sys/stat.h>
#include <sys/types.h>
#include <iostream>
#include <fstream>
#include <unistd.h>
using namespace std;
int main() {
    char filename[10] = "x";
    for (int i = 1; i <= 3; i++) {
      filename[1] = '0' + i;
      filename[2] = '\0';
      int status = mkdir(filename, 0755);
      cout << "status: " << status << endl;
      status = chdir(filename);
      cout << "status: " << status << endl;
      ofstream f("yo!");
      f << "hello" << endl;
      f.close();
      status = chdir("..");
      cout << "status: " << status << endl;
    }
  return 0;
}
```

Net effect of the program: _____

3. (5 points) What does the following command do?

_____

```
kill -15 <pid>

kill -9 <pid>
```

4. (5 points) What happens

```cpp
#include <iostream>
#include <thread>
using namespace std;

void f() {
    cout << "hello" << endl;
}

void g() {
    cout << "world" << endl;
}

int main() {
    thread t1(f);
    thread t2(g);
    t1.join();
    t2.join();
    return 0;
}
```

What is printed: _____

5. (5 points) Fill in the missing line to send a short message.

```cpp
#include <sys/socket.h>
int main() {
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    const char* msg = "hello";
    ________(sock, msg, 5, 0);
}
```

Is the message guaranteed to be sent?

Is the message guaranteed to be received?

6. (5 points) What condition indicates an error from `recv()`?

```cpp
int n = recv(sock, buf, 1000, 0);
```

7. (5 points) What is received by the server in the following code?

Client:

```cpp
    int n = send(sock, "ABC", 3, 0);
    n = send(sock, "DEF", 3, 0);
```

```cpp
char buf[1000];
int n = recv(sock, buf, 1000, 0);
```

What is n assuming:

The sends fail? _____ The sends succeed? _____

8. (5 points) Messages on a network are sent in chunks called _____.
If you send a message broken down into pieces, are they guaranteed to be received in order? _____
If you send a message via socket, are the bytes received guaranteed to be the same as the bytes sent? _____

9. (10 points) Fill in the missing code so the program sends the entire buffer.

```cpp
int sent = 0;

while(sent < len) {

    int n = send(sock,
                 buf + sent,
                 len - sent,
                 0);

    if(n < 0)
```

```
        return -1;

    __________;
}
```

10. (10 points) Fill in the missing code so the program receives a full message.

```
int received = 0;

while(received < len) {

    int n = recv(sock,
                 buf + received,
                 len - received,
                 0);

    if(n < 0)
        return -1;

    __________;
}
```

11. (8 points) What does the following IP address represent?

    127.0.0.1

    Short answer.

12. (8 points) Which protocol guarantees reliable delivery?

    - UDP
    - TCP

13. (10 points) A TCP client sends a message.
    The client later receives an **ACK** packet.
    Does this guarantee that the server application has processed the message?

    - Yes
    - No

14. (10 points) What layer generates the ACK packet?

    - Application
    - TCP
    - IP

15. (12 points) Consider the following server code.

```c
int client = accept(server_fd, NULL, NULL);

char buf[100];

recv(client, buf, 100, 0);

printf("%s\n", buf);
```

Is it guaranteed that the full message was received?

- Yes
- No

Brief reason:

16. (12 points) What is wrong with the following code?
(single phrase)

```c
char buf[100];

int n = recv(sock, buf, 100, 0);

printf("%s\n", buf);
```

Answer:

17. (15 points) Write a loop that reads from a socket until the connection closes.
Use:

- recv
- break when recv returns 0

18. (10 points) Write a program to open a file and write a multiplication table using a nested loop.

19. (10 points) Write a program that creates a subprocess using fork(). The parent should print "parent" and the child should print "child".

20. (10 points) Write a program that creates 2 threads and prints "hello" and goodbye 10 times.